



TUT02-CC2



Instructions

Coding Challenge 2: Reproducible reports on GitHub

What to submit: one **Group_#_CC2.Rmd** and its knitted **Group_#_CC2.html** per group (upload both to **TUT02-CC2** or **TUT03-CC2**, your section's folder), plus one coding-log **.zip** from positpal for each group member (upload to **CC2 – Coding Log (individual)**). Every group member also needs their own public GitHub repository containing the assignment files; the links go in the *Group members* list of the document. Details at the end of this page.

All assignments in this course are completed in groups. However, each group member should work through the assignment **INDIVIDUALLY** to generate a coding log – and for this challenge, to create their own GitHub account and repository. A recommended strategy is to set up GitHub and your repository individually during available class time, then meet during tutorial to review each other's repositories and agree on the group's document. Pay attention to how different group members organized their project folders and named their files: there is rarely one single 'right' way, and seeing the alternatives is part of the learning.

Background

This challenge continues with the *Solanum pimpinellifolium* dataset from Coding Challenge 1, so the same goals apply: **SDG 7** (Affordable and Clean Energy), **SDG 12** (Responsible Consumption and Production) and **SDG 13** (Climate Action). Putting the analysis in a version-controlled, public repository is part of the science: reproducible, shareable code is how results like these reach plant breeders, policy-makers and other researchers, and how they can check the work.

Take a moment to reflect on everything you learned in Weeks 1 and 2. It really is impressive! Now it's time to show off your skill to the rest of the world. In this assignment you'll create your own repository on GitHub, and then you can start to populate it with the assignments for this course.

A repository, sometimes shortened to just repo, is just a place to store your code. GitHub is a website that makes it easy to create and manage a repository. It uses a tool called git, which we can use to interface with GitHub and Positron.

Git and GitHub are useful for sharing code and collaborating on projects, which you'll do later in the course. But you can also think of GitHub as an online CV or Resume, which you can use to showcase your coding contributions. For example, here is a link to my GitHub site: ColauttiLab (github.com).

By now you should have read the Version Control chapter (Chapter 6) and the R Markdown chapter (Chapter 5) in the R Crash Course. Note that Chapter 6 shows the Git setup in RStudio; the steps below are the Positron equivalents.

Part I: Set up Git and your GitHub account

1. **Make a (free) account on GitHub.** Sign up at github.com. You will

need to choose a user name — pick one you would be happy to put on a CV. Add a headshot and fill out the other details of your profile. This will help your account stand out from the crowd, and make it easier for recruiters to get to know you.

2. **Install Git** if it is not already on your computer (Positron uses the Git that is installed on your system):
 - **Windows:** download and install Git from git-scm.com. Accept the defaults.
 - **macOS:** open the Terminal app and type `git --version`. If Git is missing, macOS offers to install the command-line developer tools; accept.
 - **Linux:** install `git` from your package manager.
3. **Confirm Positron can see Git.** Restart Positron, then open the **Source Control** pane (the branching icon in the left-hand activity bar, or `View → Source Control`). If Positron cannot find Git it says so here; otherwise you will see a button to initialize or open a repository.
4. **Tell Git who you are** (once per computer). In Positron open a terminal (`Terminal → New Terminal`) and run, with your own name and the email address you used for GitHub:

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Part II: Create a repository and link it to Positron

1. **Make a new GitHub repository.** Go to your GitHub site (e.g. `github.com/your_user_name`), click **Repositories** near the top of the screen, then the green **New** button.
 - Give your repository a meaningful name, like `BIOL343_CC`.
 - Give a short description.

- Choose **Public**.
 - As in Chapter 6, tick **Add a README file** and choose **R** under *Add .gitignore*. The README is the front page of your repository; you will edit it in Part III.
 - You may select a license if you would like to protect your code. You should do this when writing original code, but you don't have to do it for the assignments.
 - Click **Create repository**, then copy the full web address of your new repository (e.g. `https://github.com/your_user_name/BIOL343_CC`).
2. **Clone it into Positron.** In Positron choose **File** → **New Folder from Git...**, paste the repository address, and choose where on your computer to put it (inside your BIOL343 folder is a good choice). Positron downloads the repository (just the README and .gitignore so far) and asks whether to open it — say yes. The folder that opens is your project folder for this challenge; the repository name is also the folder name.
 3. **Look at the Source Control pane.** This is Positron's version of the RStudio *Git tab* in Chapter 6. When you start it will be empty. As you add and change files in the project folder, they appear here as *Changes*. The same three commands from Chapter 6 (*Git Commands*) apply:
 - **Pull** — fetches the latest commits from GitHub (the ... menu → *Pull*, or the **Sync Changes** button). Do this every time you start work on a project, so that you are working with the latest version.
 - **Commit** — like pressing 'save', except that each commit gets a unique code (its *SHA*) and a message, which makes it possible to roll back to any previous version. Hover over *Changes* and click + to stage the files (the *Staged* check box in Chapter 6), type a short message in the box (e.g. `Add CC2 template and data`), and

click **Commit**.

- **Push** — sends your commits to GitHub (the **Sync Changes** button, or ... → *Push*). Do this whenever you finish working for a while, and before tutorial, so your group can see your version. Try it now with the README, exactly as Chapter 6 does: open `README.md`, add a header line such as `# BIOL343_CC`, save, watch the file appear under *Changes*, then commit and push it. Refresh your repository page on GitHub to see the change.
4. **Sign in when asked.** The first time you push, Positron opens a browser window for you to sign in to GitHub and authorize Positron. After that it remembers you.

Part III: Add the assignment to your repository

Now it's time to practise the coding skills you learned last week together with the R Markdown tricks from this week, and show them to the public. Your repository will contain the data, the R Markdown file, and the knitted report.

1. **Download the template from OnQ** (BIOL343-CC2-template.Rmd) and save it in your project folder, then **rename it** (e.g. `Group_#_CC2.Rmd`); your coding log is tied to the file name. Create a `data` folder and copy `A1_Data_Excel.csv` into it from your Coding Challenge 1 project (or download it again from OnQ). Create an `images` folder for Section 6.
2. **Start your coding log** by following the **DO THIS FIRST** block at the top of the template, before you write any code (details below).
3. **Work through the template.** Every section names the chapter and section of the R Crash Course it exercises — all of it is code that appears in this week's chapters (5 and 6) or the earlier ones (3, 4, 7): the `YAML output:` block with a floating table of contents and numbered sections; ordered lists with sub-items, bold text and

links; the three chunk options `eval`, `echo` and `include`; a hand-written Markdown table; a LaTeX equation; `aggregate()` → `kable()` with a caption; an embedded `ggplot()` histogram with facets, labels and a theme; an embedded image with a relative path; and, from Chapter 6, an edited `README.md` and the SHA of your final commit.

4. Knit as you

go — `rmarkdown::render("Group_#_CC2.Rmd")` in the Console (with your file's name). Check, in the knitted file, that each chunk shows or hides exactly what its instruction asks for, and open the `.html` in a web browser to confirm the figure and image are visible: the **Render** button in Positron can write figures to a separate folder, in which case the uploaded file shows broken images.

5. Commit and push as you go. Your repository should end up containing at minimum:

the `.Rmd`, `data/A1_Data_Excel.csv`, `images/` with your species photo, and the knitted `.html`. Everything the document needs must be in the repository, so that anyone who clones it can knit it.

6. Finish as a group. In tutorial, compare your repositories and documents, agree on one version of the document, and make sure the *Group members* list links to every member's repository and that the final-commit SHA is filled in. Knit the final version and upload **both** `Group_#_CC2.Rmd` and `Group_#_CC2.html` (your group number in place of #).

Your coding log (positpal)

The routine is the same as Coding Challenge 1, and the commands are in the template under **DO THIS FIRST** and **DO THIS LAST**. Copy and paste them into the R Console; they are not code chunks and must not be added as code chunks.

With your project folder open in Positron (so that it is your working directory) and the template saved there under the name you will upload, start your log **before you write any code**, replacing `FILENAME.Rmd` with that file name:

```
install.packages("remotes")
remotes::install_url("https://quantitative.bio/downloads/positpal.tar.gz", up
positpal::start("FILENAME.Rmd")
```

Check that autosave is on in Positron (*File* → *Auto Save*). The first time you use positpal on a computer, `start()` will ask you to run `positpal::accept_eula()` once; do that, then run the `start()` line again. Run the `start()` line again every time you sit down to work on the file, and check after a few minutes that the `.capture/snaps` folder in your project is filling up.

If `start()` refuses to record, run `positpal::doctor("FILENAME.Rmd")` and bring the output to tutorial or office hours. positpal records every file in the project folder, so keep only this challenge's files in it.

When you are finished, each group member runs:

```
positpal::stop_capture()
positpal::export_capture()
```

`export_capture()` prints the name and location of the my-capture-YYYYMMDD-HHMMSS.zip it creates beside your `.Rmd`; upload that `.zip` to **CC2 – Coding Log (individual)**.

If `export_capture()` fails, zip the `.capture` folder yourself and upload that instead. Do **not** commit the `.capture` folder or the `.zip` to GitHub (add a `.capture/` line to your `.gitignore`); they are yours and OnQ's, not part of the public project.

Stuck on an error? Run `positpal::coach_brief()` and paste the text it prints into the course Coding Coach (OnQ → Course Help) along with your question.

Submission

- **One per group:** upload `Group_#_CC2.Rmd` and `Group_#_CC2.html` to your section's folder — **TUT02-CC2** (section 002) or **TUT03-CC2** (section 003). The *Group members* list must link to each member's repository, and each repository must contain the files listed in Part III step 5. Submitting to the wrong section's folder, or with a group you were not assigned to, carries the penalty described in the syllabus.
- **One per person:** upload your `my-capture-...zip` to **CC2 — Coding Log (individual)**. Upload the `.zip`, not your `.Rmd`.
- Coding challenges are graded 0–5 for completeness. A good-faith attempt that shows errors and troubleshooting earns full credit; a file that is missing, empty, or clearly not your own work does not.

Friendly reminder: to avoid losing marks, be sure to include the full names and student IDs of each group member at the top of your submitted document. You may complete this assignment individually if you have accommodations or extenuating circumstances.

BIOL343-CC2-template.Rmd (6.51 KB)

A1_Data_Excel.csv (2.25 KB)

Hydroponic_g13.jpg (1.54 MB)

Submissions

No submissions yet. Drag and drop to upload your assignment below.

Drop files here, or click below!